

Analysis and Verification of V2X Message Signatures according to ETSI TS 103 097

ABSTRACT

Introduction

Vehicle-to-Everything (V2X) communication is a key technology for future intelligent transportation systems (ITS). By enabling vehicles to exchange information with other vehicles and infrastructure, V2X aims to improve road safety, traffic efficiency, and driving comfort.

Because V2X messages can directly influence vehicle behavior, security is a critical requirement. In particular, messages must be authenticated to ensure that they originate from legitimate sources and have not been modified during transmission. For this reason, digital signatures are a fundamental security mechanism in V2X communication systems.

The European Telecommunications Standards Institute (ETSI) defines the security architecture for ITS communications in the ETSI TS 103 097 standard. This standard specifies how V2X messages are protected using cryptographic signatures based on elliptic curve cryptography.

The objective of this application project is to analyze and understand the signature mechanism of V2X messages.

This includes examining real V2X traffic captured in packet capture (pcap) files, identifying secured packets, and attempting to verify digital signatures using Python-based tools.

2. Theoretical Background

2.1 Vehicle-to-Everything (V2X) Communication

Vehicle-to-Everything (V2X) communication refers to the exchange of information between vehicles and other entities such as other vehicles (V2V), roadside infrastructure (V2I), and backend systems (V2N). The main goal of V2X communication is to increase road safety and traffic efficiency by enabling cooperative awareness and early warning of hazardous situations. V2X systems are designed to operate in highly dynamic and decentralized environments. Messages are typically broadcast periodically or event-driven, and must be processed in real time. As a result, low latency and high reliability are essential requirements for V2X communication.

2.2 V2X Protocol Stack and the Absence of TCP/IP

V2X communication differs fundamentally from traditional IP-based networks. While TCP/IP is designed for stable, point-to-point communication between known endpoints, V2X communication operates in a highly dynamic and decentralized environment.

Vehicles continuously enter and leave the communication range, and the set of receivers for a message is generally unknown at the time of transmission. Messages are typically broadcast to all vehicles within a geographic area rather than sent to a specific destination.

TCP is therefore not suitable for V2X communication due to its connection-oriented nature, handshake procedures, retransmissions, and congestion control mechanisms. These features introduce latency that is unacceptable for safety-critical applications, where messages must be processed within milliseconds and often become obsolete shortly after transmission.

As a result, the ETSI ITS protocol stack uses a connectionless communication model based on broadcast transmission. GeoNetworking provides geographic addressing and forwarding, while application-layer messages such as CAM and DENM are transmitted without establishing any prior connection.

2.3 Cryptographic Foundations of V2X Signatures

The security of V2X message authentication is based on digital signatures using the Elliptic Curve Digital Signature Algorithm (ECDSA). ECDSA relies on the mathematical properties of elliptic curves defined over finite fields and provides strong security with relatively small key sizes, which is well suited for embedded automotive systems.

An elliptic curve used in cryptography is defined by an equation of the form:

$$y^2 = x^3 + ax + b \pmod{p}$$

where p is a large prime number and all computations are performed modulo p .

The set of points (x, y) satisfying this equation, together with a special point called the *point at infinity*, forms a mathematical group.

This group has the following important properties:

- **Closure:** the sum of two points on the curve is also a point on the curve
- **Associativity:** point addition is associative
- **Identity element:** the point at infinity acts as a neutral element
- **Inverse element:** for every point p , there exists a point $-p$ such that $p + (-p) = O$, where O is the neutre

These properties make elliptic curves suitable for cryptographic constructions based on group theory.

Each vehicle in V2X communication possesses an elliptic curve key pair.

The **private key** d is a randomly chosen integer in the range $[1, n-1]$, where n is the order of the curve.

The corresponding **public key** Q is computed as:

$$Q = d * G$$

where G is a predefined **generator point** on the elliptic curve.

The generator point G is a fixed point defined by the cryptographic standard (e.g., SECP256R1 in ETSI TS 103 097) and has very important properties:

- It lies on the elliptic curve
- It generates a large cyclic subgroup

Multiplication by G means repeated elliptic curve point addition:

$$dG = G + G + \dots + G, \quad d \text{ times}$$

The fundamental security assumption of elliptic curve cryptography is the **Elliptic Curve Discrete Logarithm Problem (ECDLP)**.

While computing the public key $Q = d \cdot G$ from the private key d is computationally efficient, the inverse problem: determining d given only Q and G , is computationally infeasible.

This difficulty arises because there is no known efficient algorithm to solve the discrete logarithm problem on elliptic curves. In other words, given two points G and Q , finding the scalar d such that $Q = d \cdot G$ would require an exhaustive search over an astronomically large key space.

For a 256-bit elliptic curve, this would require approximately 2^{256} operations, which is far beyond the capabilities of any existing or foreseeable computing system. This mathematical asymmetry is what ensures the security of elliptic curve cryptography.

Before signing a message m , which is the message that the sender wants to send, it computes a cryptographic hash value:

$$e = \text{SHA-256}(m) \quad e = \text{SHA256}(m)$$

A cryptographic hash function maps input data of arbitrary length to a fixed-length output. SHA-256 produces a 256-bit hash value and has the following properties:

- **Deterministic:** the same input always produces the same output
- **Preimage resistance:** it is infeasible to reconstruct the input from the hash
- **Collision resistance:** it is infeasible to find two different inputs with the same hash
- **Avalanche effect:** a small change in the input produces a completely different hash

In contrast to simple operations such as XOR (explained in theoretical IT security lessons), SHA-256 performs multiple rounds of non-linear transformations, bitwise operations, and modular additions, making it cryptographically secure.

Hashing ensures that the signature depends on the entire message while keeping the computational complexity independent of the message length.

To generate an ECDSA signature, the sender performs the following steps:

1. Compute the hash value $e = \text{SHA-256}(m) \quad e = \text{SHA256}(m)$
2. Select a fresh random integer k , called the ephemeral key
3. Compute the elliptic curve point:

$$R = k * G$$
4. Define:

$$r = R_x \pmod{n}$$

where R_x is the x-coordinate of point R

5. Compute:

$$s = k^{-1}(e + d * r) \pmod{n}$$

The resulting digital signature is the pair (r, s)

The value k must remain secret and must never be reused, as reuse of k would allow recovery of the private key d .

2.3.1 Signature Verification Process

Upon receiving a signed message, the receiver performs signature verification using the sender's public key Q , the message m , and the signature (r,s) .

The receiver executes the following operations:

1. Compute the hash value $e = SHA256(m)$

2. Compute:

$$w = s^{-1} \bmod n$$

3. Compute

$$u_1 = e * w \bmod n$$

$$u_2 = r * w \bmod n$$

Compute the elliptic curve point:

$$X = u_1 * G + u_2 * Q$$

The signature is considered valid if:

$$x_x \bmod n = r$$

If this condition holds

The receiver can be confident that:

- the message was generated by the holder of the private key d
 - the message has not been modified during transmission
- This guarantees both **authenticity** and **integrity** of the V2X message.

2.3.2 Demonstration of Why ECDSA Verification Works

At first glance, the ECDSA verification process may appear almost magical.

However, its correctness follows directly from the mathematical structure of the elliptic curve group.

During the generation, the sender computes:

$$s = k^{-1}(e + d * r)$$

Rearranging this equation yields:

$$k = (e + d * r) * s^{-1}$$

During verification, the receiver computes

$$u_1 = e * s^{-1}, u_2 = r * s^{-1}$$

And then evaluates:

$$X = u_1 * G + u_2 * Q$$

Substituting $Q = d * G$, we obtain:

$$X = (e * s^{-1}) * G + (r * s^{-1}) * (d * G)$$

$$X = (e + d * r) * s^{-1} * G$$

$$X = k * G$$

Thus, the computed point X , is identical to the original point R , generated during the signing process. Since both points share the same x-coordinate, the verification condition $X_2 = r$.

2.4 Security Architecture for V2X Communications Based on PKI

In V2X communication, messages are exchanged over an open wireless medium. Any nearby device can receive transmitted data, which makes traditional security mechanisms such as passwords, prior connections, or interactive authentication impractical. Vehicles continuously enter and leave the communication range, and messages must be processed and trusted immediately after reception. For this reason, the main security challenge in V2X systems is not confidentiality, but authentication: determining whether a received message was actually sent by a legitimate and authorized vehicle.

This problem is addressed through the use of a Public Key Infrastructure (PKI). Rather than relying on direct trust relationships between vehicles, the PKI provides a common trust framework that all participants can rely on. In simple terms, the PKI defines who is allowed to participate in the system, which cryptographic keys are considered trustworthy, and how this trust can be verified by receiving vehicles. When a message is received, the PKI allows the receiver to answer essential questions such as who issued the public key used to sign the message, whether that key can be trusted, and whether the sender is authorized to transmit V2X messages.

The PKI does not verify messages by itself. Instead, it enables meaningful verification by introducing trusted entities known as Certificate Authorities (CAs). These authorities are responsible for issuing and digitally signing certificates. Each vehicle participating in the V2X system is equipped with one or more cryptographic key pairs, consisting of a private key and a corresponding public key. Vehicles typically have more than one key pair in order to preserve privacy. Instead of using a single long-term identity, vehicles are provisioned with many short-term pseudonymous certificates, each containing a different public key. These certificates are changed regularly to prevent long-term tracking of individual vehicles.

The private key associated with each certificate is securely stored inside the vehicle and is used to sign outgoing messages using the Elliptic Curve Digital Signature Algorithm (ECDSA). The public key, however, is never distributed on its own. It is always embedded inside a digital certificate issued by a trusted Certificate Authority.

It is important to clearly distinguish between certificates and signature verification, as these are two related but different concepts. A certificate establishes trust in a public key, while signature verification uses that trusted public key to authenticate a message. A digital certificate is a structured data object that binds a public key to an authorized participant in an Intelligent Transportation System (ITS), such as a vehicle or a roadside unit. In addition to the public key, the certificate contains validity information, usage permissions, and other metadata, as well as a digital signature generated by a trusted CA. By verifying the CA's signature on the certificate, a receiving vehicle can trust that the public key contained in the

certificate belongs to a legitimate V2X participant. Only once this trust is established does the public key become useful for message authentication.

When a vehicle transmits a secured V2X message, it signs the message using its private key via ECDSA. The message either includes the sender's certificate directly or references a certificate that has been transmitted previously. Upon reception, another vehicle obtains the sender's certificate, extracts the public key, and uses it to verify the digital signature. If the verification is successful, the receiver can be confident that the message was generated by a vehicle possessing the correct private key and that the message has not been modified during transmission. This process is one-way and non-interactive, meaning that no handshake or prior connection is required, which is essential in the highly dynamic V2X environment.

Unlike traditional Internet PKI systems, V2X PKI is explicitly designed with privacy in mind. Vehicles do not rely on a single long-term identity certificate. Instead, they use large sets of short-lived pseudonymous certificates that are periodically changed. This approach ensures that messages can be authenticated and trusted while preventing vehicles from being easily tracked over long periods of time.

Within this framework, ECDSA plays a central role and is used for two distinct purposes. First, Certificate Authorities use ECDSA to sign certificates, thereby vouching for the authenticity of the public keys they contain. Second, vehicles use ECDSA to sign outgoing V2X messages using the private key associated with a certificate. ECDSA is particularly well suited for V2X systems because it provides strong cryptographic security with relatively small key sizes, low computational overhead, and reduced message size, which is crucial in bandwidth-constrained wireless environments.

The ETSI TS 103 097 standard defines how these cryptographic mechanisms are applied to ITS messages in European V2X systems. Rather than introducing new algorithms, the standard specifies how existing primitives such as PKI and ECDSA are used in practice. It defines the structure of secured ITS messages, how digital signatures are applied, how certificates are included or referenced, and how receivers verify message authenticity. A secured packet, as defined by the standard, wraps an original ITS message, such as a CAM or DENM, together with security-related information.

A secured V2X message is therefore not a single opaque block of data, but a structured object composed of several logical elements. It contains the payload, which carries the original ITS application data, and the to-be-signed (TBS) data, which specifies exactly which bytes of the message are protected by the digital signature. The signature is computed exclusively over the TBS data, not over the entire packet, and any modification to fields included in the TBS will cause signature verification to fail. This structure is clearly visible in analysis tools such as Wireshark, where the payload, TBS data, and signature appear as separate components.

Conceptually, when a vehicle receives a secured V2X message, it should extract the TBS data, compute a cryptographic hash, obtain the sender's public key from the certificate, verify the ECDSA signature, and finally validate the certificate and its trust chain. In this project, only the first four steps could be fully implemented. The final step, which involves

validating the certificate chain and trust anchors, could not be completed due to missing issuer certificates and restricted access to PKI repositories.

This application project does not aim to implement ETSI TS 103 097 in its entirety. Instead, it focuses on understanding and verifying selected core aspects of the standard in practice, particularly the structure of secured packets, the extraction of TBS data, and the verification of ECDSA signatures using public keys obtained from certificates.

This conceptual understanding of PKI, certificates, and ETSI TS 103 097 forms the foundation for the implementation, results, and evaluation presented in the following sections.

In this application project, ETSI TS 103 097 is not implemented in its entirety. Instead, selected aspects of the standard are analyzed and verified in practice. In particular, the focus is placed on understanding the structure of secured packets, extracting the to-be-signed data, and verifying ECDSA signatures using public keys obtained from certificates.

This conceptual understanding of PKI and ETSI TS 103 097 forms the basis for the practical implementation and evaluation presented in the following sections.

3 Concept and Design

This application project did not start from a predefined system architecture, but from a concrete technical question that emerged during the analysis of real captured V2X traffic. While inspecting secured V2X messages, it was not immediately clear how digital signatures were applied in practice, nor which parts of a packet were actually protected by the signature.

The main question that motivated this work was therefore the following: **which part of a secured V2X packet is covered by the digital signature, and which data is actually protected against modification?**

Given a secured message, it was initially unclear which fields were included in the signature computation and which fields could theoretically be modified without invalidating the signature. Understanding this distinction was essential before attempting any practical signature verification.

3.1 Identifying the Signed Data in V2X Messages

To answer this question, the ETSI TS 103 097 standard was consulted. The standard defines the concept of *To-Be-Signed Data (TBS)*, which specifies exactly which bytes of a secured V2X message are included in the digital signature (depending on the document version and formatting, this can typically be found around pages 30–35 of the ETSI TS 103 097 v1.3.x specification).

Rather than signing the entire packet, ETSI TS 103 097 defines a structured security envelope where only security-relevant fields are covered by the signature. This design allows flexible message handling while still guaranteeing integrity and authenticity of the critical data.

More precisely, the digital signature covers exclusively the content contained inside the `tbsData` structure of the `signedData` element. The analysis of real captured packets confirmed that the following fields are included in the signature computation:

- `tbsData.payload`, which contains the original ITS application message (e.g., CAM or DENM). This ensures that the complete application-layer content cannot be modified without invalidating the signature.
- `tbsData.headerInfo`, including fields such as the PSID and the generation time. These parameters are protected in order to guarantee the integrity of the contextual metadata required for correct message interpretation.

In addition, although the sender's certificate is not formally part of the TBS structure itself, the signature input defined by the standard includes the hash of the certificate together with the hash of the TBS data. Consequently, any modification to the certificate would also invalidate the signature.

Conversely, all fields located outside the TBS structure are not protected by the digital signature. This includes the signature field itself, the signer information wrapper, and

lower-layer networking headers such as GeoNetworking, BTP, and IEEE 802.11. These elements can be processed or adapted by the communication stack without affecting signature validity, provided that the TBS content remains unchanged.

However, due to the complexity and length of the standard, it was not immediately obvious how this theoretical definition of TBS data maps to real packet captures. For this reason, a practical, experiment-driven approach based on real captured V2X messages was chosen instead of relying solely on the standard documentation.

3.2 Verification Strategy

Once it was understood that ETSI TS 103 097 relies on ECDSA signatures computed over the TBS data, a concrete verification strategy was defined. The goal was to replicate, as closely as possible, the verification process performed by a receiving vehicle.

The verification process consists of extracting the TBS data from a captured secured V2X packet, computing a cryptographic hash over this data, extracting the ECDSA signature components (r , s), reconstructing the sender's public key from the certificate, and finally verifying the signature using standard cryptographic libraries.

This approach directly reflects the conceptual verification process defined in the standard and allows independent validation using external tools.

3.3 Extension towards PKI Validation

After successfully verifying the digital signature over the To-Be-Signed (TBS) data, the scope of the project was naturally extended. At this point, the verification was mathematically correct, but an important question remained: *can the sender of the message actually be trusted?*

For this reason, the objective was no longer limited to checking whether the signature was valid, but to better understand the **complete authentication process** defined by the V2X Public Key Infrastructure (PKI).

This extension involved working with certificate-related data observed in captured packets, reconstructing elliptic curve public keys from their compressed binary format, and using those keys to verify signatures. Through this process, it became clear that cryptographic verification and PKI are closely linked: while cryptography proves that a signature is correct, PKI is what gives meaning and trust to the public key used in that verification.

3.4 Dependency on Certificates and PKI

From a purely mathematical point of view, verifying an ECDSA signature only requires a public key. However, this alone is not enough to establish trust. Without additional context, there is no way to know who owns that public key or whether it belongs to a legitimate vehicle.

This is where the PKI comes into play. In simple terms, the PKI acts as a system of trust that links public keys to authorized vehicles. In the V2X context defined by ETSI TS 103 097, vehicles do not generate their own trusted identities. Instead, they receive certificates from trusted authorities.

Vehicles use short-term certificates (also called pseudonym certificates), which are issued by dedicated authorities. These authorities are themselves trusted because their certificates ultimately link back to a Root Certificate Authority (Root CA). If a receiver trusts the Root CA, it can also trust certificates issued further down the chain.

At this stage of the project, the design goal expanded: in order to fully validate trust, it would be necessary to obtain the relevant CA certificates and verify the certificate chain. This is what motivated the investigation of the European CPOC platform.

3.5 Attempt to Obtain PKI Material from CPOC

The CPOC platform (<https://cpoc.jrc.ec.europa.eu>) is operated by the European Commission and is intended to support experimentation with Cooperative Intelligent Transport Systems (C-ITS). It was expected to provide example PKI material, including CA certificates used in European V2X deployments.

However, during the execution of the project, it became apparent that this platform no longer provides complete and downloadable PKI datasets. Only limited documentation and partial references were available, and no full CA certificate chain could be retrieved.

As a result, this approach reached a dead end from a PKI validation perspective. Instead of stopping the project, this limitation motivated a deeper investigation into alternative sources of trust information, including binary files that appeared indirectly during the experiments.

3.6 Analysis of the OER File Structure

As part of this alternative approach, OER-encoded files referenced during the experiments were analyzed. Before describing this analysis in detail, it is useful to briefly clarify some concepts. ASN.1 (Abstract Syntax Notation One) is a formal language used to describe the structure of complex data objects, such as certificates or secured messages. Encoding rules such as OER (Octet Encoding Rules) define how these abstract structures are translated into raw binary data that can be transmitted or stored.

As a first step, an attempt was made to decode the OER file using generic ASN.1 decoding tools. This attempt failed, which was both expected and technically meaningful. OER encoding is strictly schema-dependent, and ETSI TS 103 097 defines complex and highly specific ASN.1 structures. Without access to the official ETSI ASN.1 schemas, semantic decoding of the file is not possible. This confirmed that the file could not be interpreted as a complete certificate or trust store using generic tools alone.

Following this, the analysis shifted to a low-level inspection of the raw binary content of the OER file. Instead of attempting to interpret the data semantically, the file was examined directly in hexadecimal form. This type of inspection makes it possible to identify embedded human-readable ASCII strings that may appear within otherwise binary data.

During this raw inspection, a readable ASCII identifier was directly observed inside the binary content of the file:

EU-ROOT-CA-L2

This identifier was not obtained through successful ASN.1 decoding, but was instead visible as plain text within the raw byte stream of the OER file. Such identifiers are commonly included in certificate-related data structures as internal labels or naming fields used by certificate authorities.

The string itself is meaningful in the context of European PKI infrastructures. The prefix “EU” suggests a European-level authority, “ROOT CA” refers to a root Certificate Authority, and “L2” typically denotes a second-level or intermediate CA under a root authority. Taken together, this strongly suggests that the file is related to a European V2X root or intermediate certificate authority, rather than to an individual vehicle or pseudonym certificate.

This observation provides strong evidence that the analyzed OER file contains PKI trust material, such as root or intermediate CA information. However, without the corresponding ASN.1 schemas and issuer certificates, it is not possible to fully decode the file or to determine the exact semantic role of the contained data. The analysis therefore confirms the presence of cryptographically relevant trust material, while also highlighting the limitations imposed by missing schema and PKI context.

3.7 Targeted Search for Cryptographic Material

At this point, several facts were clear. ETSI TS 103 097 mandates the use of ECDSA over the NIST P-256 elliptic curve, and every CA certificate must therefore contain an elliptic curve public key. Even if full ASN.1 decoding fails, this cryptographic material must still physically exist in the file.

Based on this reasoning, a targeted heuristic approach was adopted. Two custom scripts were developed to search the OER file for compressed elliptic curve public keys and for issuer identifiers observed in real V2X traffic. The specific focus was on two hashedId8 values repeatedly observed in secured messages:

```
ae2d8d9dc6165ddf  
0498fbf3b8b8c249
```

The analysis confirmed the presence of cryptographically valid elliptic curve public keys within the file, indicating that it contains material consistent with a CA trust store. However, without full ASN.1 decoding, the exact semantic role of these keys (e.g., root or intermediate CA) could not be determined.

4. Implementation

4.1 Overview of the Implementation Approach (Including PKI Aspects)

The implementation phase of this project focused on experimentally validating the security mechanisms defined in ETSI TS 103 097 by working directly with real captured V2X traffic. Rather than implementing a full ITS security stack, the objective was to analyze existing secured packets and verify their cryptographic properties using independent tools.

The implementation was intentionally divided into incremental steps. First, the correctness of ECDSA signature computation was verified at a purely cryptographic level. This step answers the question of whether the signature mathematically matches the transmitted data and the sender's public key. Second, the role of certificates and the PKI was investigated to understand how cryptographic verification is extended into trust establishment.

From an implementation perspective, it is important to distinguish between **cryptographic verification** and **PKI validation**. Cryptographic verification checks whether a signature is mathematically correct with respect to a given public key. PKI validation, on the other hand, determines whether this public key is authorized and trusted by the system. While cryptographic verification can be performed locally using only packet data, PKI validation depends on external trust anchors and certificate chains.

Due to the limited availability of PKI material (e.g., CA certificates), the implementation focuses on demonstrating cryptographic verification and analyzing the structural challenges that prevent full PKI validation in practice.

4.2 Initial Signature Verification Script (verifier_quick_check.py)

The purpose of the first script was to determine **which exact data is covered by the digital signature** in secured V2X messages and to verify the correctness of the signature computation.

At this stage, the goal was **not** to validate the full PKI chain (later on we explain about it), but to confirm the cryptographic integrity of the message.

4.2.1 Extraction and Preparation of Input Data

The script starts by defining the hexadecimal representations of the main elements extracted from the packet capture:

```
tbs_hex      = "4003808201672040..."
r_comp_hex   = "b223cdec3b081aa3..."
s_comp_hex   = "a3b7e8d7794eba79..."
cert_hex     = "80030080ae2d8d9d..."
pub_com_hex  = "f2b6dcbb166fef54..."
```

These values correspond to:

- `tbs_hex`: the To-Be-Signed Data extracted from the secured message
- `r_comp_hex`, `s_comp_hex`: the ECDSA signature components
- `cert_hex`: the sender certificate bytes
- `pub_com_hex`: the compressed public key coordinate extracted from the certificate

All values are converted into byte representations before further processing.

Since elliptic curve public keys are transmitted in compressed form, the script reconstructs the full public key point by testing both possible compression prefixes:

```
pub_prefix = b'\x03'
pub_bytes = pub_prefix + pub_com
pubkey = ec.EllipticCurvePublicKey.from_encoded_point(
    ec.SECP256R1(), pub_bytes
```

)

If reconstruction fails, the alternative prefix (0x02) is tested. This step is required because the parity of the y-coordinate is not always explicitly known.

The ECDSA signature component r is derived from the x-coordinate of the elliptic curve point $R = k \cdot G$. Wireshark reports this value in compressed form, which must be converted back into an integer:

```
R_prefix = b'\x02'
R_bytes = R_prefix + r_comp
Rpub = ec.EllipticCurvePublicKey.from_encoded_point(
    ec.SECP256R1(), R_bytes
)
r_int = Rpub.public_numbers().x
```

If reconstruction fails, the script falls back to interpreting r directly as an integer. This reflects the ambiguity sometimes encountered in real packet captures.

According to IEEE 1609.2 and ETSI TS 103 097, the actual input to the signature algorithm is constructed as:

```
hd = hashlib.sha256(tbs).digest()
hs = hashlib.sha256(cert_der).digest()
message = hd + hs
```

This confirms that the digital signature is computed over the hash of the TBS data concatenated with the hash of the certificate.

Finally, the signature is encoded in DER format and verified:

```
der_sig = encode_dss_signature(r_int, s_int)
pubkey.verify(der_sig, message, ec.ECDSA(hashes.SHA256()))
```

A successful verification demonstrates that:

- the extracted TBS data is correct
- the signature corresponds to the provided public key
- the message integrity is preserved

However, this verification remains purely cryptographic and does not validate certificate trust.

4.3 Extended Verification Script (`verify_packet73.py`)

The second script was developed to overcome the limitations identified in CAM messages. Its objective was to perform a complete cryptographic verification using a certificate directly embedded in the packet, enabling a more realistic validation scenario.

Unlike the first script, the second implementation explicitly tests both compression prefixes when reconstructing the public key:

```
def try_reconstruct_pubkey(pub_com):
    for prefix in (b'\x02', b'\x03'):
        try:
            pb = prefix + pub_com
            pk = ec.EllipticCurvePublicKey.from_encoded_point(
                ec.SECP256R1(), pb
            )
            return pk, prefix
        except Exception:
            pass
```

This approach increases robustness when handling real-world packets where the compression format may vary.

The second script also introduces a more advanced strategy for reconstructing the signature value `r`. Several possible interpretations are tested:

```
def try_reconstruct_r(r_raw):
    candidates = []
    candidates.append(("raw_int", int.from_bytes(r_raw, "big")))

    if len(r_raw) == 33 and r_raw[0] == 0x04:
        candidates.append(("strip_04", int.from_bytes(r_raw[1:],
"big"))))

    if len(r_raw) == 32:
        for pref in (b'\x02', b'\x03'):
            try:
                Rpub = ec.EllipticCurvePublicKey.from_encoded_point(
                    ec.SECP256R1(), pref + r_raw
                )
                candidates.append(("point",
Rpub.public_numbers().x))
            except Exception:
                pass
    return candidates
```

Each candidate is tested until a valid signature verification is obtained.

The final verification loop attempts all reconstructed variants:

```
for r_name, r_int in r_cands:
    der_sig = encode_dss_signature(r_int, s_int)
    pubkey.verify(der_sig, message, ec.ECDSA(hashes.SHA256()))
```

A successful result confirms that:

- the message was signed using the private key corresponding to the certificate
- the certificate's public key was correctly reconstructed
- the signature is cryptographically valid

4.3 Implementation and Analysis Workflow

The implementation phase started with the analysis of the file **B78E9AF02C26C465.oer**. At the beginning of the project, it was not clear what this file actually contained. Several hypotheses were considered, including the possibility that it stored a CA public key, a full CA certificate, a root or intermediate trust anchor, or auxiliary PKI-related metadata.

According to ETSI TS 103 097, all V2X certificates and security-related structures must be encoded using ASN.1 with Octet Encoding Rules (OER). In this context, “mandates” simply means that the standard explicitly requires this encoding format. Based on this requirement, it was reasonable to assume that the file contained PKI-related security structures, even if their exact content and structure were unknown.

4.3.1 First Script: ASN.1 Decoding Attempt and Raw Inspection

The goal of the first script was exploratory. It aimed to determine whether the OER file could be decoded using standard ASN.1 tools without having access to the official ETSI TS 103 097 schemas.

The script began by loading the file as raw binary data:

```
with open(file_path, 'rb') as f:
    oer_data = f.read()
```

A minimal and generic ASN.1 schema was then defined and compiled using the `asn1tools` library:

```
compiled = asn1tools.compile_string(schema, codec='oer')
decoded = compiled.decode('MyData', oer_data)
```

This decoding attempt was intentionally simplistic. The goal was not full semantic decoding, but rather to test whether the file could be interpreted as a generic ASN.1 structure. As expected, the attempt failed with a decoding error.

This result is technically meaningful. OER encoding is strictly schema-dependent, and ETSI TS 103 097 defines complex and highly specific ASN.1 structures. Without the exact schema definitions, semantic decoding is not possible. This confirmed that the file could not be decoded using generic ASN.1 tools and that a different analysis strategy was required.

After the decoding attempt failed, the script switched to a low-level inspection of the file contents. The file was treated purely as raw bytes. First, a hexadecimal dump of the initial bytes was produced:

```
print(binascii.hexlify(oer_data[:100]).decode('ascii'))
```

In addition, the script scanned the file for printable ASCII sequences:

```
for byte in oer_data:
    if 32 <= byte <= 126:
        text_data += chr(byte)
```

An ASCII string is simply a sequence of bytes corresponding to readable characters. Their presence inside a binary file often indicates embedded identifiers or metadata.

During this inspection, readable strings were found inside the file. One particularly relevant string was:

EU-ROOT-CA-L2

This identifier is significant because it closely matches naming conventions used in European Union PKI infrastructures. “ROOT CA” typically refers to a root certificate authority, while “L2” usually denotes a level-2 intermediate CA. This strongly suggests that the file contains European ITS trust anchor material rather than an end-entity vehicle certificate. However, without the ASN.1 schema, the file could still not be converted into a usable certificate object.

4.3.2 Motivation for a Second Script

At this point, several practical limitations were clear. Semantic ASN.1 decoding was not feasible without official ETSI schemas, OpenSSL does not support OER decoding, and official EU PKI repositories (such as CPOC) were no longer publicly accessible. As a result, certificates referenced by captured V2X packets could not be retrieved.

Despite these limitations, cryptographic verification fundamentally requires public keys, not full certificate semantics. This led to a key insight: even if ASN.1 decoding fails, the raw elliptic curve public keys must still exist physically inside the OER file.

This realization motivated the development of a second, more targeted script.

4.3.3 Second Script: ECC Public Key and hashedId8 Scanning

The design of the second script was guided directly by ETSI TS 103 097. The standard mandates ECDSA using the NIST P-256 elliptic curve, and compressed public keys on this curve have a well-defined binary format of exactly 33 bytes. In addition, certificates are often referenced indirectly using `hashedId8` identifiers.

Two issuer identifiers repeatedly observed in captured V2X packets were provided by the project supervisor:

```
ae2d8d9dc6165ddf
0498fbf3b8b8c249
```

These identifiers were configured in the script as search targets:

```
TARGET_HASHES = [
    "ae2d8d9dc6165ddf",
    "0498fbf3b8b8c249"
]
```

The script scanned all `.oer` files in the working directory and searched for these `hashedId8` values. When a match was found, a window of surrounding bytes was extracted to provide local context:

```
outname, a, b = dump_window(fname, data, off, win=WINDOW)
```

Next, the script scanned the entire file for compressed ECC public key candidates. These are identified by a prefix byte of `0x02` or `0x03`, followed by 32 bytes:

```
if bdata[i] in (2, 3):
    cand = bdata[i:i+33]
```

Each candidate was then tested by attempting to reconstruct it as a P-256 public key:

```
pub = ec.EllipticCurvePublicKey.from_encoded_point(
    ec.SECP256R1(), comp_bytes
)
```

The cryptography library automatically verifies the mathematical correctness of the elliptic curve point. Invalid candidates fail reconstruction and are rejected without additional checks.

This approach is not blind brute force, but a heuristic search based on strict cryptographic constraints defined by the standard

4.3.4 Interpretation of the Results

The second script identified several byte sequences that could be reconstructed as valid elliptic curve public keys. This confirms that the analyzed OER file contains cryptographic material consistent with CA certificates or trust anchors.

However, the presence of multiple valid public keys does not imply that multiple independent CA keys exist. In ASN.1-encoded certificates, the same public key may appear multiple times in different fields, such as verification key indicators or authorization structures. In addition, ASN.1 OER encoding contains length fields, bit strings, and flags that can coincidentally resemble valid ECC points when interpreted in isolation.

As a result, while valid cryptographic keys were clearly detected, their exact semantic role within the PKI could not be determined without full ASN.1 decoding. This confirms the presence of meaningful PKI material while also highlighting the practical limitations of low-level analysis in the absence of official schemas and complete certificate chains.

5. Results and Evaluation

The main objective of this project was to understand how secured V2X messages are authenticated according to ETSI TS 103 097, and to evaluate how far this verification can be performed in practice using real captured traffic.

At the message level, the verification process was successfully implemented. Using captured CAM packets, it was possible to correctly extract the to-be-signed data (TBS) from secured V2X messages and to reconstruct the cryptographic hash exactly as defined by the standard. This step proved to be the core of the entire verification process, since only the TBS is actually signed. Any mistake in identifying the correct TBS bytes would inevitably lead to failed or misleading verification results.

Once the TBS was correctly extracted, the ECDSA signature of the message could be verified using the public key contained in the sender's certificate. This confirms that the secured packet structure was correctly interpreted and that the cryptographic verification mechanism defined in ETSI TS 103 097 works as intended when the required information is available. In practical terms, successful verification demonstrates that the message was not modified in transit and that the sender possessed the private key corresponding to the public key included in the certificate.

In addition to message-level verification, the project analyzed the structure of certificates observed in real V2X traffic. The analysis showed that certificates follow the ETSI-defined structure, consisting of a toBeSigned part and an associated ECDSA signature. Public keys were encoded as compressed ECDSA P-256 keys, and issuer information was not transmitted explicitly, but referenced indirectly using hashedId8 identifiers. This behavior is consistent with the design goals of V2X systems, which aim to reduce bandwidth usage by avoiding the transmission of full certificate chains in every message.

The ECDSA signature protecting the certificate itself could also be identified, confirming that certificates are signed by an issuing authority. However, full verification of these certificate signatures could not be completed. The issuer public keys referenced by the observed hashedId8 values were not available in the captured data, nor could they be retrieved from publicly accessible repositories. As a result, while the structure and presence of certificate signatures were confirmed, their cryptographic validation could not be finalized.

The analysis of OER files further reinforced these observations. Although semantic decoding was not possible without official ASN.1 schemas, raw inspection revealed strong indicators of PKI trust material, such as identifiers consistent with European V2X root or intermediate certificate authorities. The detection of multiple valid elliptic curve public keys confirmed that the files contain cryptographically meaningful data. However, without schema information and issuer certificates, it was not possible to assign clear roles to these keys or to reconstruct a complete certificate hierarchy.

Overall, the evaluation shows that V2X message verification is reliable at the cryptographic level, provided that the TBS is correctly extracted and the sender's public key is available. The main limitation encountered was not incorrect verification or false positives, but rather the absence of external information required for full PKI validation. This reflects the reality of

operational V2X systems, where trust is established through pre-installed infrastructure and external repositories rather than through isolated packet analysis.

6. Conclusion and Future Work

6.1 Conclusion

This application project investigated the security mechanisms of V2X communications with a focus on digital signatures, certificates, and Public Key Infrastructure (PKI) as defined in ETSI TS 103 097.

The project demonstrated that secured V2X messages can be analyzed and verified at the cryptographic level using real captured traffic. By correctly extracting the to-be-signed data from secured packets and applying ECDSA verification, it was shown that message authenticity and integrity can be validated using standard cryptographic libraries. This confirms that the security model defined by ETSI is not only theoretical, but also practically applicable.

Beyond message verification, the project provided insight into the structure and usage of V2X certificates in real deployments. The separation between message signatures and certificate signatures was clearly identified, and the indirect referencing of certificate authorities through hashedId8 identifiers was shown to be a central design element of the V2X PKI model.

The investigation of OER-encoded files further highlighted that trust material in V2X systems is distributed in specialized binary formats that are not easily interpreted using generic tools. While cryptographic material such as elliptic curve public keys could be detected, full semantic interpretation of certificates and trust anchors was not possible without official ASN.1 schemas and issuer certificates.

A key conclusion of this project is that cryptographic correctness alone is not sufficient for full PKI validation. Although signatures and public keys may be mathematically valid, their trustworthiness depends entirely on external infrastructure and pre-established trust relationships. This reflects a fundamental principle of V2X security: vehicles are designed to operate within a trusted ecosystem rather than independently reconstructing certificate chains from observed traffic.

Overall, the project achieved its objectives by combining theoretical understanding with hands-on analysis and by providing a realistic view of both the capabilities and the limitations of V2X PKI verification in practice

6.2 Future Work

Several directions for future work emerge naturally from this project. A major extension would be the implementation of full ASN.1 OER decoding using the official ETSI TS 103 097 schemas, which would allow semantic decoding of OER files and direct extraction of certificates and trust anchors.

With access to issuer certificates referenced by hashedId8 identifiers, full certificate chain verification could be implemented, enabling validation up to a trusted root CA. Integrating analysis tools with official ETSI or EU PKI repositories would address the current limitation of missing issuer certificates and enable real-world trust validation.

Additional future work could include automated certificate caching, lifetime management, and support for certificate revocation mechanisms such as CRLs. Finally, further analysis of privacy aspects, including pseudonym rotation and linkability risks, would provide valuable insight into the balance between security and privacy in operational V2X systems.

6.3 Final Remarks

This project shows that V2X security is not only a cryptographic challenge, but also a systems and infrastructure problem. Correctly interpreting secured messages requires an understanding of how standards, certificates, and trust infrastructure interact in real deployments.

By combining packet-level analysis, cryptographic verification, and PKI investigation, this work provides a solid foundation for further academic research or practical development in the field of intelligent transportation systems.

PACKAGES ANALIZED:

PACKAGE 73

Frame 73: Packet, 594 bytes on wire (4752 bits), 594 bytes captured (4752 bits) on interface Ath 1 (not configured), id 0

Prism capture header

802.11 radio information

IEEE 802.11 QoS Data, Flags:

Logical-Link Control

GeoNetworking

Basic Header

0001 = Version: 1

.... 0010 = Next Header: Secured Packet (2)

Reserved: 0x00

Life Time: 5

0000 01.. = Life Time multiplier: 1

.... ..01 = Life Time base: 1 s (1)

Remaining Hop Limit: 1

[Expert Info (Note/Sequence): "Remain Hop Limit" only 1]

["Remain Hop Limit" only 1]

[Severity level: Note]

[Group: Sequence]

Secured Packet

ieee1609Dot2Data

protocolVersion: 3

content: signedData (1)

signedData

hashId: sha256 (0)

tbsData

payload

data

protocolVersion: 3

content: unsecuredData (0)

unsecuredData [...]:

20500280008a0100140076f9cc6bbba8dc6f7f61d0b8c02058e500f8000082b0000a00007d1
00000202cc6bbbbbafaae405a560ea04e1b0441e24023881e3f4f905882b120007e02968a873
7fee9fff200407f941980105602ed809f98ce01f5f035ac0c28c800032f81ab2066

headerInfo

psid: psid-ca-basic-services (36)

generationTime: 2025-04-22 07:58:41.005621 (672393526005621)

signer: certificate (1)

certificate: 1 item

Item 0

Certificate

version: 3

type: explicit (0)

issuer: sha256AndDigest (0)
 sha256AndDigest: 0498fbf3b8b8c249
toBeSigned
 id: none (3)
 none: NULL
 cracId: 000000
 crlSeries: 0
 validityPeriod
 start: 2025-04-22 03:00:00 (672375605)
 duration: hours (4)
 hours: 168
 appPermissions: 2 items
 Item 0
 PsidSsp
 psid: psid-ca-basic-services (36)
 ssp: bitmapSsp (1)
 bitmapSsp: 010000
 Item 1
 PsidSsp
 psid: psid-den-basic-services (37)
 ssp: bitmapSsp (1)
 bitmapSsp: 01901a25
 verifyKeyIndicator: verificationKey (0)
 verificationKey: ecdsaNistP256 (0)
 ecdsaNistP256: compressed-y-1 (3)
 compressed-y-1:
5a7d745952e03581f8d1b415eb417f91e932d69f3678be402d8a13dd1ca9a191
 signature: ecdsaNistP256Signature (0)
 ecdsaNistP256Signature
 rSig: x-only (0)
 x-only:
0dfd997091c10b634a6c4e753f088416ea3de69c911ae8866d28c48dedcf59d3
 sSig:
01ecd7c71d75e9afe3caf5c5d5a51e371b3aed754dd34c25268b3b646e31e445
 signature: ecdsaNistP256Signature (0)
 ecdsaNistP256Signature
 rSig: compressed-y-0 (2)
 compressed-y-0:
08e0dd964f9acd6cdfd0d0bf57f49af34dcd15664fd4b55e79e514d302106938
 sSig:
939e47bbecedbaf18d8736511d114fe57b90a9d0cec278fd52cbda4dc204d009
Common Header
 0010 = Next Header: BTP-B Transport protocol (2)
 0000 = Reserved: 0x0
 Header type: Topologically-scoped broadcast Single-hop broadcast (SHB) (0x50)
 Traffic class: 2
 0... = Store Carry Forward: 0
 .0.. = Channel offload: 0

..00 0010 = Traffic class ID: ITS-G5 Access Category Best effort (2)

Flags

1... = Mobility flag: 1

.000 0000 = Reserved: 0

Payload length: 138

Maximum Hop Limit: 1

Reserved: 0x00

Topologically-Scoped Broadcast Packet

Source position: 140076f9cc6bbba8dc6f7f61d0b8c02058e500f8000082b

GN_ADDR: 140076f9cc6bbba

0... = Manual: 0

.001 01.. = ITS-S type: Passenger Car (5)

.... ..00 0000 0000 = ITS-S Country Code: Reserved (0)

MID: 76:f9:cc:6b:bb:ba (76:f9:cc:6b:bb:ba)

Timestamp: 2378627062ms

Latitude: 48d43'11,48"N (487296002)

Longitude: 9d19'16,57"E (93212687)

1... = Position accuracy indicator: 1

.000 0000 0000 0000 = Speed: 0,00 m/s

Heading: 209,1 degrees

Decentralized Congestion Control - Multi Channel Operation

Local channel busy ratio: 0,00 % (0)

Max neighbouring CBR: 0,00 % (0)

1010 0... = Output power: 20 dBm

.... .000 = Reserved: 0

Reserved: 0

BTP-B

Intelligent Transport Systems

References

ETSI TS 103 097 V1.3.1 — Intelligent Transport Systems (ITS); Security; Security header and certificate formats. ETSI specification defining the security header, certificate formats, and hashedId8 usage in V2X communication.

ETSI TS 103 097 V1.2.1 — Intelligent Transport Systems (ITS); Security; Security header and certificate formats. Provides normative definition of HashedId8 and SignerInfoType, including how certificate digests are computed and referenced.

ETSI TS 103 097 (Release 2/Latest) — Specification summary from ETSI describing secure ITS message and certificate formats, including support for explicit and implicit certificates.

V2X Security Overview — Secure V2X PKI and Certificate Usage. General V2X security context and certificate requirements (referenced by Nexus documentation).

V2X Security Knowledge Base — NFinity. Describes region- and standard-based V2X security stacks (Europe: ETSI TS 103 097).

PKI and Certificate Chain Usage in V2X (5GCAR D4.1). Discussion of hashed certificate identifiers, certificate exchange and revocation lists in V2X systems.

Vanetza Certify Tool Documentation. Example of how ETSI TS 103 097 certificate generation and PKI integration can be conducted for test V2X PKI setups.

IEEE Std 1609.2 — Standard for Wireless Access in Vehicular Environments (WAVE) Security Services. Though not directly fetched online, this is a normative reference cited by ETSI TS 103 097 for common security primitives and certificate handling.